RMFS Simulator Documentation

Contents

1 Assumptions **2**

2 General Description **2**

3 Code Structure **2**

4 Input Parameters **3**
4.1 Order Generation . . . 3

5 Domain Entities **5**
5.1 Order . . . 5
5.2 Pod . . . 5
5.3 Robot . . . 5
5.4 Workstation . . . 5
5.5 Task and Visit . . . 6
5.6 Enumerations . . . 6
5.7 Warehouse . . . 6
5.7.1 Warehouse Layout . . . 6

6 Simulator Engine **8**
6.1 `SimulatorConfig` — immutable parameters . . . 8
6.2 `SimulatorState` — mutable per-run state . . . 8
6.3 `Simulator` — orchestrator . . . 8
6.4 Event Handlers . . . 9
6.5 Greedy Policies . . . 9
6.5.1 Assignment Order-Workstation . . . 9
6.5.2 Task Design and Sequencing . . . 11

7 KPIs Collection **12**

1 Assumptions

For the sake of simplicity, the simulator is based on the following simplifying assumptions.

- Replenishment missions are not considered, therefore no inventory constraints are modelled. Each pod contains a limited set of SKUs, but the exact quantity or volume of each item is not known.
- Technical aspects of the robots, such as energy management and low-level routing, are outside the scope of this simulator and are not modelled.
- At the end of a mission, the pod will return to its storage position.

2 General Description

The discrete-event simulator emulates the operational dynamics of an Amazon-style RMFS (Robotic Mobile Fulfillment System), based on a *parts-to-picker* approach: robots bring pods to workstations, where operators pick items and process orders.

The simulator supports two decision-making modes:

1. **Cyclic optimisation:** every Δt_{opt} a mathematical model is solved for task generation and sequencing.
2. **Greedy policies:** online decisions are made based on policies with limited lookahead.

In both modes, the assignment of a task to a robot is always managed via a greedy policy based on the minimization of the distance between the resting position of the robot and the storage location of the pod used by the task.

3 Code Structure

The project is organised on four logical levels: `core` (shared entities and data structures), `sim` (DES engine and event handlers), `opt` (assignment policies) and `stat` (KPIs cellection management).

Path	**Role**
`run_simulation.py`	Entry point: configures logging, builds the warehouse, starts the simulation
`config.py`	Centralised numerical parameters
`scripts/core/enums.py`	Enumerations for discrete states and event types
`scripts/core/entities.py`	Dataclasses for `Order`, `Pod`, `Robot`, `Workstation` and `Task`.
`scripts/core/queues.py`	Generic priority queue (min-heap with O(1) lookup)
`scripts/core/warehouse.py`	Physical layout and generation, distance utilities
`scripts/sim/Simulator.py`	Main DES engine: clock, event loop, dispatch table
`scripts/sim/event_handler.py`	Handler function for each event type
`scripts/opt/policies.py`	Greedy assignment policies
`scripts/stat/StatManager.py`	Main collector of KPIs
`scripts/stat/core.py`	Dataclasses to support the collection of KPIs

Table 1: Folder and file structure of the project.

4 Input Parameters

The following table lists the simulator configuration parameters. The parameters that define order generation and pod processing times at workstations have a domain defined by [BJS24][1].

The numerical values present are those adopted by [BJS24] or fixed based on the literature; parameters marked with `???` are yet to be defined.

Parameter	**Domain**	**Description**
Layout and capacity		
`NUM_PODS`	???	Total number of pods (`GRID_ROWS` × `GRID_COLS`)
`NUM_SKUS`	???	Number of distinct SKUs
`NUM_SKUS_PER_POD`	20	Target number of distinct SKUs for each pod
`NUM_ROBOTS`	???	Number of autonomous mobile robots
`NUM_WORKSTATIONS`	???	Number of picking stations
`GRID_ROWS / GRID_COLS`	??? / ???	Pod grid dimensions
`WS_ORDER_CAPACITY`	3	Maximum number of open orders per station
`WS_RELEASED_TASK_CAPACITY`	???	Number of tasks released per workstation
Operational times		
`ROBOT_SPEED`	30.0	Robot speed (cells/min, ≈ 0,5 m/s)
`POD_PROCESS_TIME`	5 s	Processing time of a pod at the workstation
`ITEM_PROCESS_TIME`	5 s	Picking time of an item at the workstation
Order generation		
`INTERARRIVAL_TIME_ORDER`	$\left\{\frac{12}{35}, \frac{6}{25}, \frac{3}{10}, \frac{3}{40}, \frac{1}{5}, \frac{3}{50}, \frac{3}{100}, \frac{3}{250}\right\}$	Mean order inter-arrival time (min)
`PROB_1_ITEM_ORDER`	0.5	Probability that an order contains a single item
`GEO_DIST_PARAM_ORDER`	{0.25, 0.35, 0.45, 0.65}	Geometric parameter for multi-item orders
Simulation control		
`TIME_HORIZON`	???	Total simulation duration (min)
`DELTA_T_OPT`	15 min	Optimiser call interval

Table 3: Numerical parameters of the simulator divided by functionality.

The order generation algorithm is reported here for completeness, as it operationalizes the stochastic rules defined by the parameters table.

4.1 Order Generation

Order generation follows the model of Barnhart et al [BJS24]:

[1] GitHub repository `https://github.com/alexschmid3/warehouse-task-assignment`.

Algorithm 1 Order Generation

```
Sample u ~ Uniform(0, 1)
if u < PROB_1_ITEM_ORDER then
    order_size ← 1
else
    Sample g ~ Geometric(p) with p = GEO_DIST_PARAM_ORDER
    order_size ← g + 2
end if
for i = 0 to order_size − 1 do
    Sample SKU_i from a truncated normal distribution centered in N/2 and with standard deviation N/6
    ▷ N is the total number of unique SKUs in the warehouse
end for
```

5 Domain Entities

Entities are implemented as Python dataclasses and represent the state of the system during the simulation. The main ones are: `Order`, `Pod`, `Robot`, `Workstation`, `Task` and `Visit`.

5.1 Order

Tracks a customer order from arrival to completion.

Field	Type	Description
`order_id`	`int`	Unique identifier
`arrival_time`	`float`	Order arrival time
`order_size`	`int`	Number of items required
`items_required`	`list[int]`	Full list of required items
`items_pending`	`list[int]`	Items not yet picked (decremented as items are picked)
`workstation_id`	`int | None`	Assigned workstation (`None` if not yet assigned)
`status`	`OrderStatus`	Lifecycle state: `BACKLOG → WAITING → OPEN → CLOSED`

5.2 Pod

Represents a storage pod. When idle, the pod is at its `storage_location`; when on a mission, its status becomes `BUSY`, and the location is not known.

Field	Type	Description
`pod_id`	`int`	Unique identifier
`storage_location`	`int`	Identinfier of the storage cell
`items`	`list[int]`	SKUs contained in the pod
`status`	`PodStatus`	`IDLE` or `BUSY`

5.3 Robot

Each robot is always in one of two states: `IDLE` (available) or `BUSY` (executing a task).

Field	Type	Description
`robot_id`	`int`	Unique identifier
`position`	`id`	Current position in the grid in terms of cell id (the information is known only when the robot is not moving)
`status`	`RobotStatus`	`IDLE` or `BUSY`

5.4 Workstation

The workstation manages two separate buffers: one for pods (`picking_buffer`) and one for orders waiting to be opened (`order_buffer`). Note that the `status` refers to the picking operation, not to whether any orders are currently open at the workstation.

Field	Type	Description
workstation_id	int	Unique identifier
order_capacity	int	Maximum simultaneously open orders
released_task_capacity	int	Maximum released tasks per workstation
position	int	Cell identifier of the position
pod_process_time	float	Pod processing time at a workstation
item_process_time	float	Time for a picking operation at a workstation
opened_orders	set[int]	Orders currently being processed
order_buffer	list[int]	Orders waiting to be opened
picking_buffer	list[int]	Pods waiting (excluding the active one) for a picking operation
active_tasks	set[int]	IDs of released tasks allocated to a robot
released_tasks	set[int]	IDs of released tasks not yet allocated
status	WorkstationPickingStatus	IDLE or BUSY

5.5 Task and Visit

A `Task` is a mission assigned to a pod-robot pair, composed of an ordered sequence of stops (`stops`). Each stop is a `Visit`, which specifies the destination workstation, the orders served (`orders`), and the items to be picked (`items`). The `priority` field of the task determines the dispatching order in the queue.

5.6 Enumerations

`OrderStatus`, `RobotStatus`, `PodStatus` and `WorkstationPickingStatus` define the discrete states of the entities, making state transitions explicit in the simulation logic.

5.7 Warehouse

The `Warehouse` class collects all information and state variables of the system entities. In particular, its attributes include the lists of pods, workstations, and robots.

This class also provides methods to generate the warehouse layout based on the number of pods along the x-axis, the number of pods along the y-axis, the number of workstations and the number of robots.

5.7.1 Warehouse Layout

The warehouse layout is generated from `grid_rows` and `grid_cols`. Pods are arranged in a rectangular grid; after every two columns of pods, an aisle of width two cells is inserted to allow robot movement. The resulting physical dimensions are:

$$X = \text{grid_rows} + 2\left\lfloor \frac{\text{grid_rows} - 1}{2} \right\rfloor + 2 \cdot \text{MARGIN} - 1, \qquad Y = \text{grid_cols} + 2 \cdot \text{MARGIN} - 1$$

Workstations are placed along the perimeter, preferably on the bottom side in a symmetric configuration. Robots are initialized at random, non-overlapping positions within the warehouse. The travel time between two cells is computed as the sum of two components: a nominal travel time, obtained by dividing the Manhattan distance by `robot_speed`, and a positive stochastic component accounting for potential delays due to congestion or other effects.

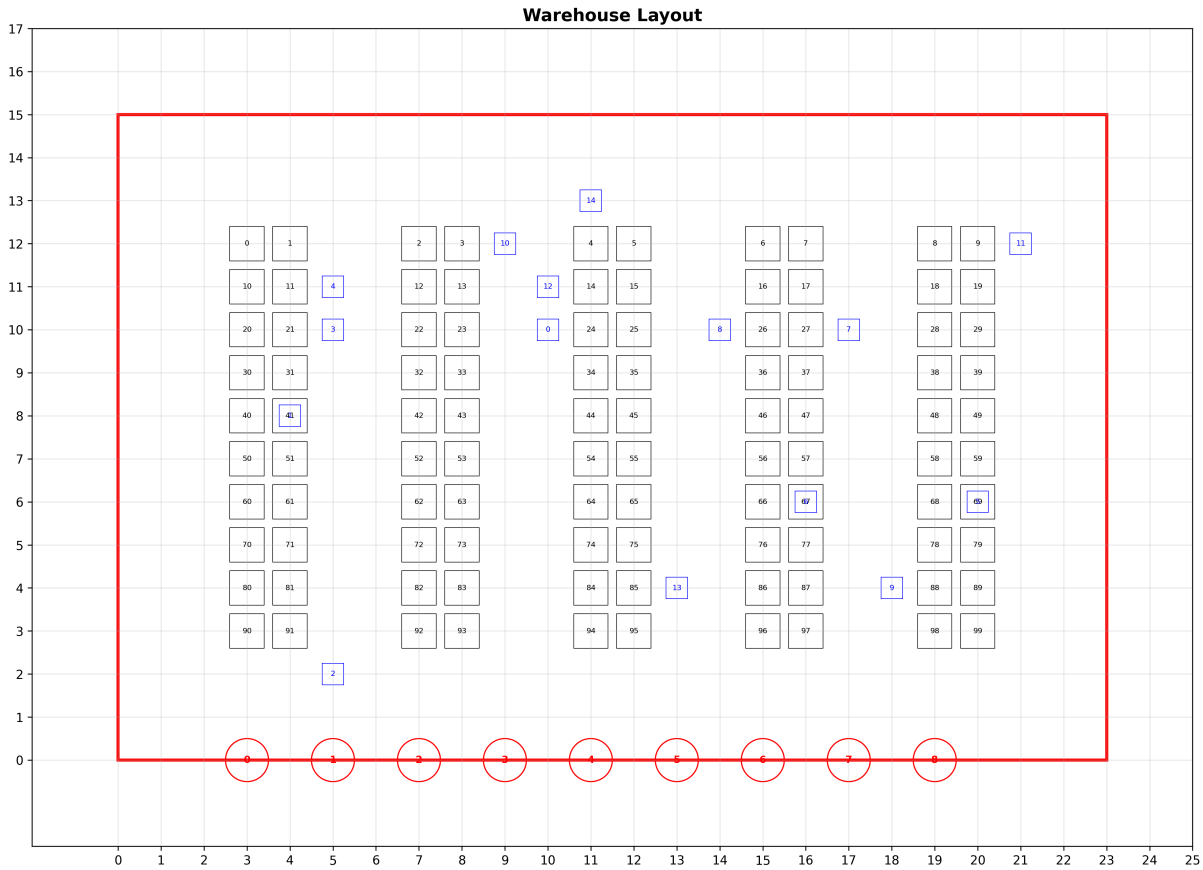

Figure 1: Example layout created with `GRID_ROWS` = 10, `GRID_COLS` = 10, `NUM_ROBOTS` = 15 and `NUM_WORKSTATION` = 9.
Black cells are the `storage_location` of each pod, red ones belong to workstations, and blue ones are the initial positions of the robots.

6 Simulator Engine

The simulation engine is split across three Python objects to cleanly separate immutable configuration from mutable per-run state.

6.1 `SimulatorConfig` — immutable parameters

A `frozen` dataclass that holds every parameter that does not change across replicas. It is constructed once and shared by all `run()` calls on the same `Simulator` instance.

Attribute	Type	Description
`order_gen_config`	`list[float]`	`[interarrival_time, prob_1_item, geo_param]` — parameters for order arrival generation
`warm_up`	`float`	Seconds discarded from statistics at run start. Default `0`.
`optimization_enabled`	`bool`	`False` → greedy policies; `True` → optimiser

6.2 `SimulatorState` — mutable per-run state

A plain dataclass holding all quantities that change during a simulation run. A fresh `SimulatorState` is constructed at the beginning of every `run()` call, guaranteeing that consecutive calls on the same `Simulator` object produce fully independent replicas with no shared mutable state.

Attribute	Type	Description
`current_time`	`float`	Current simulation time (seconds)
`warehouse`	`Warehouse`	Physical environment and live state of all entities
`future_events`	`PriorityQueue[Event]`	Future events ordered by timestamp; `RELEASE_TASK` breaks ties first
`orders_in_system`	`PriorityQueue[Order]`	Received orders ordered by arrival time; `BACKLOG` orders have priority
`scheduled_tasks`	`dict[int, PriorityQueue[Task]]`	Tasks scheduled per workstation by the optimiser
`released_tasks`	`PriorityQueue[Task]`	Tasks ready for robot assignment
`active_tasks`	`dict[int, Task]`	Executing tasks, keyed by `task_id`
`orders_counter`	`int`	Monotonic counter for order IDs
`task_counter`	`int`	Monotonic counter for task IDs
`event_count`	`int`	Total events processed (for logging)
`closed_orders`	`int`	Completed orders (for logging)

6.3 `Simulator` — orchestrator

The `Simulator` class owns the immutable configuration, the random generator and the `StatManager`. Its constructor accepts a `warehouse_factory` — a zero-argument callable that returns a fresh `Warehouse` — instead of a `Warehouse` instance directly. This design choice is intentional: calling `run()` multiple times in the main script produces independent replicas automatically, because each call rebuilds the full `SimulatorState` (including a new `Warehouse`) from scratch before processing any event.

Attribute	Type	Description
`RANDOM_GENERATOR`	`numpy.Generator`	Random generator for reproducibility
`config`	`SimulatorConfig`	Immutable run parameters
`state`	`SimulatorState`	Mutable state of the current run (rebuilt on every `run()` call)
`STAT_MANAGER`	`StatManager`	KPI collector (also rebuilt on every `run()` call)
`_warehouse_factory`	`Callable[[], Warehouse]`	Factory invoked once per `run()` to produce a pristine warehouse

The typical usage pattern in `run_simulation.py` is therefore:

```
config = SimulatorConfig(order_gen_config=[...], warm_up=3600)
```

```
sim    = Simulator(rng, config, warehouse_factory=lambda: Warehouse(...))

sim.run(8 * 3600)    # replica 1 -- starts from a clean state
sim.run(8 * 3600)    # replica 2 -- fully independent, zero extra code
```

The main event loop inside `run()` follows the classical DES structure:

1. Rebuild `state` and `STAT_MANAGER` from a fresh warehouse.
2. Push the first `ARRIVAL_ORDER` event at $t \approx 0$.
3. Pop the event with the smallest timestamp from `state.future_events`.
4. Advance `state.current_time` to the event time.
5. Dispatch to the corresponding handler via the dispatch table.
6. Repeat until the queue is empty or `TIME_HORIZON` is reached.
7. Write and reset statistics via `STAT_MANAGER`.

6.4 Event Handlers

Each `EventType` is associated with a handler function with signature `handler(event, state, sim)`, where `state` is the `SimulatorState` holding all mutable queues and counters, and `sim` provides access to the immutable configuration (`sim.config`) and the random generator (`sim.RANDOM_GENERATOR`). The table summarises the current implementation status.

Note that once a task is extracted from the priority queue in `START_TASK`, it is then passed along between events as input and moves through the simulation.

In the `Simulator` class, when the `future_events` queue is initialized, ties between simultaneous events are resolved by prioritizing `RELEASE_TASK` events. This is because the greedy policy used for task generation must consider the full set of released tasks in order to avoid producing redundant tasks, which could lead to inconsistencies in the simulation. For the same reason, at the end of the `END_PICKING` event, if a new order is opened at the workstation, no new tasks are designed or released, as this will be handled during the `OPEN_ORDER` process. The reason behind these design choices will become clearer when the greedy policies are introduced.

The following flowcharts illustrate the dependencies between events and the overall system dynamics.

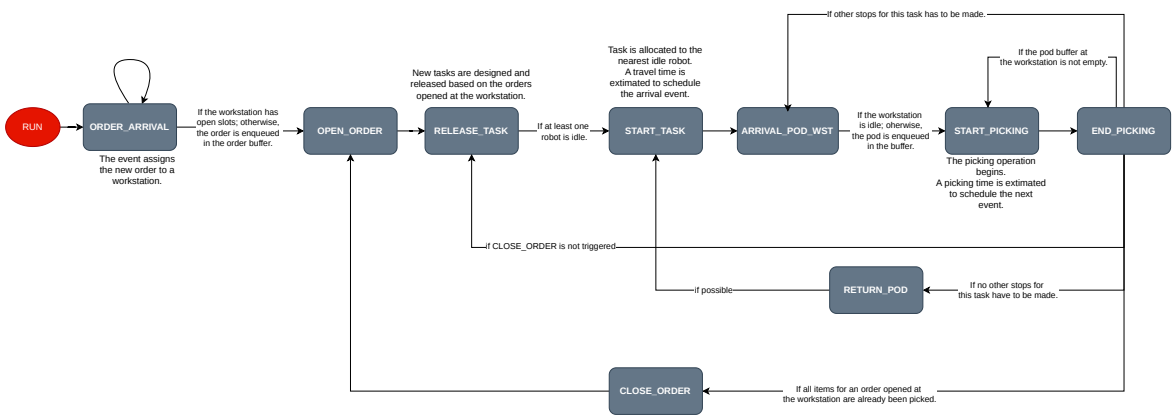

Figure 2: Graph showing the relationship between events when `optimization_enabled = False`. Arrows indicate which event will be scheduled next.

6.5 Greedy Policies

6.5.1 Assignment Order-Workstation

The assignment order-workstation is performed whenever an order arrives in the system. The greedy policy implemented is the following[2]:

In this policy, the sequencing of orders at a given workstation is neglected, as their priority is solely given by their arrival time.

[2] Vorrei migliorarla cercando di creare dei cluster in base alle sku required dalgli order alle workstation

Event	Event.info	Responsibility
ARRIVAL_ORDER	None	Generates a new order, store it in state.orders_in_system and schedules the next order arrival; without the optimiser, assigns it directly to a workstation.
OPEN_ORDER	Order	Sets the order to OPEN, inserts it into opened_orders and, if possible, generates the associated tasks and triggers RELEASE_TASK.
RELEASE_TASK	Task (without optimiser) / int	Releases tasks: directly if the optimiser is not used, otherwise retrieving them from those scheduled for the workstation; updates the workstation state.
RUN_OPTIMIZER	**TODO**	Invokes the optimiser to assign backlog orders and generate tasks.
START_TASK	None	Verify that among the released tasks there is at least one that can be started (i.e., whose corresponding pod is idle). If so, assign an idle robot (if available) to it and set both the robot and the pod to BUSY.
ARRIVAL_POD_WST	Task	Handles pod arrival at the workstation: inserts it into the picking_buffer and triggers START_PICKING if the workstation is idle.
START_PICKING	Task	Starts picking, sets the workstation to BUSY and schedules END_PICKING according to the picking process times.
END_PICKING	Task	Completes the picking operation by updating the items_pending field of the involved orders. If necessary, it triggers CLOSE_ORDER, and schedules new pod movements or their return to storage based on the remaining stops in the task. Since the number of released tasks for this workstation has been decremented, new tasks can now be created and released.
CLOSE_ORDER	Order	Closes the order when completed, frees a slot at the workstation and trigger OPEN_ORDER if possible.
RETURN_POD	Task	Returns the pod to its storage_location, sets robot and pod to IDLE and enables new tasks to start.

Algorithm 2 Assignment order-workstation

1: Select workstation_id as the open workstation minimizing the total number of orders, computed as the sum of queued and currently open orders

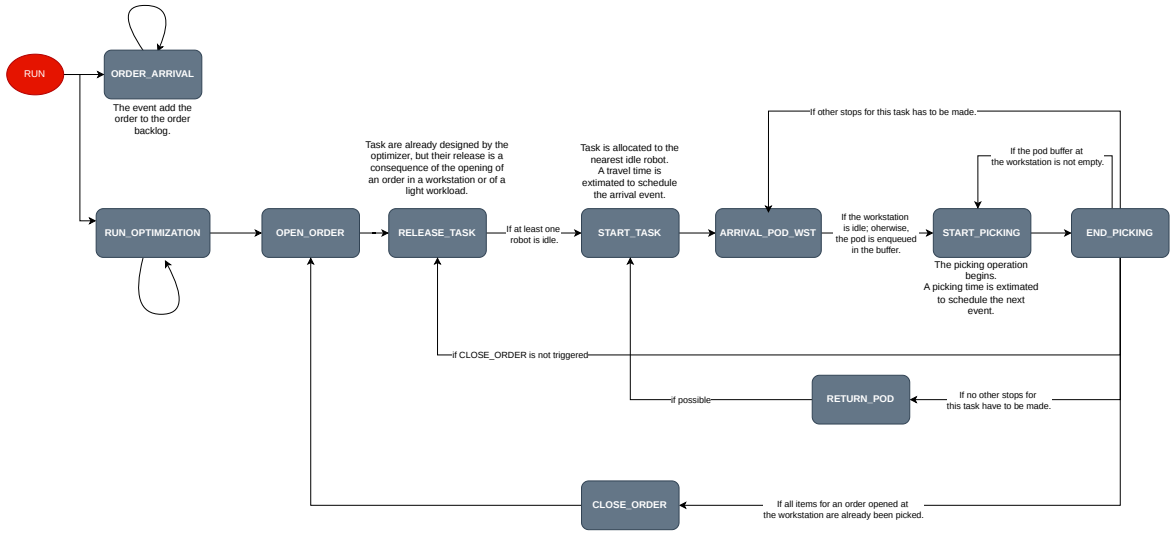

Figure 3: Graph showing the relationship between events when `optimization_enabled = True`. Arrows indicate which event will be scheduled next.

6.5.2 Task Design and Sequencing

Task design is performed greedily, considering only the orders currently open at a given workstation, with the objective of maximizing pile-on that is, minimizing the number of pods required to complete all open orders at the workstation. During the algorithm, all released tasks associated with the given workstation that have not yet been allocated are redesigned to account for newly required items. This is why the realising of already designed tasks must be prioritized in order to avoid inconsistencies.

In practice, task sequencing is neglected, as priority is based exclusively on the time at which tasks are designed and released.

Algorithm 3 Task Design

Compute `uncovered_skus` as the set of SKUs required by open orders at the workstation that are not yet covered by active tasks
if `uncovered_skus` is empty **then**
    return
end if
while the number of designed tasks is below the maximum number of tasks that can be generated **and** `uncovered_skus` is not empty **do**
    Rank pods according to the number of items in `uncovered_skus` they can serve (breaking ties by distance)
    Select the highest-ranked pod
    Define a `Visit` based on the subset of orders that benefit from the selected pod
    Create a task containing the defined `Visit`
end while
return Tasks designed

Some observations have to be made. First, this policy does not allow tasks with multiple visits to be defined, as it does not have global information about all workstations, but only about the one that triggers the policy. Second, in this policy the while loop continues until the number of designed tasks reaches the maximum number of tasks that can be generated. This is due to the fact that the number of released and active tasks per workstation is limited by `WS_RELEASED_TASK_CAPACITY`. This design choice is intended to prevent overcrowding near the workstation in scenarios where the number of robots significantly exceeds the system's processing capacity (a large fleet of robots serving a small number of slow workstations, for istance), which would otherwise lead to unnecessary congestion.

7 KPIs Collection

The main objective of this simulator is to evaluate two different optimization approaches in a controlled and reproducible environment; hence, several relevant statistics must be collected.

The collected statistics are grouped into three categories:

- `ResourceTracker`: this type of statistic measures the state frequency of a given entity over time. It is used, for example, for both robots and workstations, and captures respectively the robot utilization rate and the workstation picking time ratio.
- `OrderFlowTracker`: this statistic captures the average order flow time, grouped by order size, from the `BACKLOG` state to the `CLOSED` state.
- `TimeWeightedMeanTracker`: this type of statistic is used to compute time-averaged KPIs, such as the average number of open orders per workstation.

Finally, all statistics can be aggregated to compute global performance indicators and support system-level analysis.

References

[BJS24] Cynthia Barnhart, Alexandre Jacquillat, and Alexandria Schmid. "Robotic warehousing operations: a learn-then-optimize approach to large-scale neighborhood search". In: *ArXiv* abs/2408.16890 (2024). URL: https://api.semanticscholar.org/CorpusID:272310402.